EX11 - Taylor con funciones - Retorno multiple

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex11_taylor_exp_function.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Convertir EX07 en una funcion taylor_exp(x,N,xp) que devuelva el valor numerico y la expresion simbolica.

Variables / elementos del problema

Nombre	Significado
x	simbolo
N	orden
xp	punto de evaluacion

Ecuacion o logica en papel

```
e^x ~ sum_{n=0..N} x**n/n!
```

Comprobacion manual

La matematica es la misma que EX07; lo nuevo es llamar varias veces a la misma funcion con distintos N y xp.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- funciones
- return
- Taylor
- listas

Nuevo en este ejercicio:

- return de dos valores
- desempaquetado de retorno
- mezcla de argumentos posicionales/keyword
- reutilizacion para estudiar convergencia

3. Conceptos Python nuevos

1. Retorno multiple

Sintaxis: `return ex_numeric, expression_to_show`

La funcion devuelve dos valores. Python permite recibirlos en dos nombres separados.

2. Desempaquetado

Sintaxis: `approximation, expression = taylor_exp(...)`

El primer retorno va al primer nombre y el segundo al segundo.

3. Mezcla de argumentos

Sintaxis: `taylor_exp(x, N=5, xp=0.5)`

Los posicionales deben aparecer antes de los keyword.

4. Codigo paso a paso

Paso 1. Definir funcion

```
def taylor_exp(x, N, xp):
```

La funcion recibe todo lo necesario para construir y evaluar la aproximacion.

Paso 2. Validar N

```
if not isinstance(N, int) or N < 0:
    return None, None
```

Como el caller espera dos valores, el caso invalido tambien devuelve dos valores.

Paso 3. Devolver dos resultados

```
return ex_numeric, expression_to_show
```

Uno sirve para calculo numerico y el otro para mostrar el polinomio.

Paso 4. Recibir ambos

```
approximation, expression_to_show = taylor_exp(x, N, xp)
```

Esta linea ejecuta la funcion y desempaqueta sus dos retornos.

Paso 5. Llamar varios casos

```
taylor_exp(x, N=0, xp=0.5)
taylor_exp(x, N=5, xp=0.5)
taylor_exp(x, N=10, xp=2)
```

La funcion se reutiliza para estudiar como cambia la aproximacion sin duplicar el algoritmo.

5. Flujo de ejecucion

```
def taylor_exp
 -> validate N
 -> build polynomial
 -> return 2 values
 -> unpack
 -> compare cases
```

6. Resultado / comprobacion esperada

Para N=10 y xp=2, la aproximacion es ~7.388994709 frente a exp(2)~7.389056099. N=-1 o N=3.5 devuelven None,None.

7. Errores tipicos al empezar

- Esperar un retorno cuando hay dos.
- Poner un posicional despues de keyword.
- Olvidar devolver dos valores tambien en el error.
- Duplicar el algoritmo para cada caso.

8. Si solo recuerdas cinco cosas

- Una funcion puede devolver varios valores.
- Los retornos se desempaquetan por posicion.
- Posicionales antes que keywords.
- Reutilizar funciones facilita comparar casos.
- El algoritmo vive en un solo lugar.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.